

Slide Narratives

A rehearsal-first companion to the Cursor Meetup deck. Use it to remember the point, land the concrete example, and move cleanly to the next slide.

1. State the point

One sentence. No apology. Let the slide support you.

2. Ground it

Use the concrete repo, human navigation, or test evidence example.

3. Move forward

Each transition tells the room why the next slide exists.

SLIDE 01

Title

POINT

Every coding agent starts cold unless the repo carries its own map.

TALK TRACK

Open with the simple pain: every new agent session has to rediscover the repo. It reads files, guesses boundaries, and often misses the one invariant that would make the answer safe.

Then address the obvious objection early:

A big `AGENTS.md` works until it becomes a junk drawer. It costs context on every run, it goes stale, rules compete with each other, and there is no freshness or verification layer.

I still want `AGENTS.md`, `CLAUDE.md`, and `.cursorrules`, but I want them to be signposts, not the whole map. `agent-context` moves the map into smaller reviewable pieces: overview, invariants, search scope, completeness contracts, and freshness checks.

The talk is about turning repeated rediscovery into a small artifact that lives with the code.

TRANSITION

Before the mechanism, establish why you care about this problem and where it came from.

SLIDE 02

Who Is Talking

POINT

This comes from both production agent work and open-source agent tooling.

TALK TRACK

Introduce yourself as someone building the operating layer around AI agents: context, evals, workflows, controls, and production adoption. At work, the theme is making agents useful after the demo. In OSS, `agent-chorus` gives session evidence across agents, while `latchkeyd` explores local control and boundaries. Today is the navigation piece: `agent-context`.

TRANSITION

Now move from biography to the shared problem in the room.

SLIDE 03

The Hook

POINT

The agent is new here every time.

TALK TRACK

Frame the agent as capable but contextless. It does not know which files matter, which paths are deprecated, which invariants are dangerous, or what the last agent already learned. The cost is not just tokens or time; it is repeated uncertainty. A cold session becomes bounded work only when the repo carries a map the agent can load first.

TRANSITION

That means the fix is not only "use a better model."

SLIDE 04

Reframe

POINT

The failure is missing system evidence.

TALK TRACK

Software teams already check in tests, schemas, docs, runbooks, and CI because private memory is not enough. Agent context should be the same kind of artifact: portable, reviewable, and fresh. The standard is not magical: read the system map before acting on the system.

TRANSITION

Although code makes this problem obvious, the pattern is bigger than repos.

SLIDE 05

Generalize

POINT

Every serious system needs a navigation layer.

TALK TRACK

For this audience, the system is a code repository. But the pattern applies to any complex system with state, rules, risks, and work to do: data platforms, customer accounts, operations runbooks, and product workflows. Today we stay with code because it is inspectable and familiar.

TRANSITION

To make the pattern precise, separate it into three control surfaces.

SLIDE 06

Three-Track Architecture

POINT

Explorable recall has three separable tracks: navigation, operating loop, and engineering.

TALK TRACK

Define explorable recall as the ability for an agent to rediscover the same map, from reviewable artifacts, whenever it needs to work. Navigation asks what to load and when to stop. Operating loop asks how a specific agent consumes that context. Engineering asks what makes the artifact durable and auditable.

TRANSITION

This talk mostly focuses on one track: navigation.

Today's Session

POINT

Navigation is the control surface for what loads, what does not, and when the agent has enough.

TALK TRACK

Start with the human version. When I work in a codebase I know well, I do not re-read the whole repo. I know which file probably owns the behavior, which folder is generated, which test is the real signal, and which colleague can point me to the right subsystem. That is navigation.

Agents need the same kind of pointer. Today we often give it manually every session: "look in this folder, avoid that old path, run this test, don't touch this generated file." It works, but it burns context every time. Then the thread gets long, compaction starts, details get summarized away, and we repeat the same briefing in the next session.

So `agent-context` is not a wrapper around everything you might paste into the prompt. It is a deterministic pathway for exploration: for this kind of task, start here, search only there, verify this shortcut, and stop when the contract is satisfied.

The goal is not to teach every detail of every codebase to the model. The goal is to offload the repeatable briefing into the repo: required files, excluded directories, stop rules, and verification shortcuts. That is why the system has both human-readable markdown and machine-readable JSON.

TRANSITION

Now show the actual artifact that carries this navigation.

The Artifact

POINT

`.agent-context/` is a committed folder, not a hosted service or hidden memory.

TALK TRACK

Walk through the layers. Content gives humans and agents the overview. Authority gives trust-and-follow agents explicit routes and completeness contracts. Navigation gives search-and-verify agents scoped search and shortcuts. Quality gives verification, freshness, acceptance tests, and manifests. The important bit: it is in git, so it can be reviewed and kept fresh.

TRANSITION

The same artifact works because different agents use it differently.

Same Navigation, Opposite Loops

POINT

One pack supports two agent behaviors.

TALK TRACK

Claude-like agents often trust and follow the pack. They can stop when the completeness contract is satisfied. Cursor and Codex are more search-and-verify: they still inspect source, but the pack tells them where to search and what evidence matters. Same navigation design, different consumption pattern.

TRANSITION

So how does this get created in practice?

Workflow

POINT

Ask, author, self-test, verify, review.

TALK TRACK

The fastest path is not hand-authoring eleven files. Start in a large repo and ask the agent to use the `agent-context` skill. The skill inventories the repo, scaffolds if needed, and fills the structured pack.

The routing files stay tiny. `AGENTS.md`, `CLAUDE.md`, and `.cursorrules` should not become the full context; they should be a small signpost, roughly 100-300 tokens depending on the repo, that tells the agent how to enter `.agent-context/current/`.

Once the pack is reviewed and merged, hooks and policy keep it fresh. In agent-authored PRs, the agent includes `.agent-context` updates as a separate commit. In human-authored work, the pre-push hook and CI policy can flag drift, then someone asks the agent to update the context. `main` is the preferred sync anchor because it is the shared truth, though teams can adapt the base branch.

The important operating rule: update only the relevant context slice. Maybe a new file family was added, a data schema changed, a subsystem appeared, verification shortcuts moved, or stale guidance should be deleted. The agent treats that as a context update, not permission to edit product code without explicit approval.

On first setup, the current skill already creates grep-verified acceptance tests so it can test whether the navigation actually works. It asks lookup, impact-analysis, cross-cutting, and diagnosis questions; answers from the pack first; verifies against the real code; and if the pack misses something, it improves the relevant section and reruns the test.

In the team-skills / stronger workflow, that acceptance loop can be run as sub-agent pressure testing: separate agents try to use the pack, expose weak routes or missing ground truth, and the authoring agent tightens the context before the PR lands.

Then the CLI verifies structure and freshness. Finally, you review the generated context diff like any other code change. The key line for the room: first run is not just "write docs"; it is closer to a small research loop that creates ground truth, tests the map, and tightens weak routes.

TRANSITION

That review step matters because the artifact becomes part of the system.

SLIDE 11

Quality Gate

POINT

Review the context diff, not the model's private memory.

TALK TRACK

Show what a reviewer should look for: file families that change together, negative guidance for deprecated or generated paths, and silent failures that do not show up as compile errors. The example invariant is the kind of thing agents miss when they only browse files opportunistically.

TRANSITION

Once this artifact exists, we can test whether it changes agent behavior.

SLIDE 12

Test Protocol

POINT

There is an internal pack self-test, then an external bare-vs-structured evaluation.

TALK TRACK

There are two test layers. First, the pack tests itself during creation: lookup, impact analysis, cross-cutting impact, and diagnosis. The agent answers from the pack, verifies against grep, improves weak sections, and records the result in `acceptance_tests.md`.

Then there is the external experiment protocol. That method is intentionally boring: same repo, same task, only context changes. The tasks are multi-hop, because lookup tasks are too easy. Ground truth is grep-backed, and grading uses a fixed rubric. Structured runs are only allowed to start after verify and freshness pass.

TRANSITION

Now show the current evidence, with the caveat that it is directional.

SLIDE 13

Quantified Evidence

POINT

Structured context improved correctness across all measured lanes.

TALK TRACK

Do not overperform the numbers. Say the evidence is current, useful, and LLM-provisional. The headline is that structured context improved every lane: Claude reaches 100 percent, Cursor default gets the biggest lift, Cursor Opus medium gets much faster, and risk flags drop to zero for Codex and Cursor Opus medium.

TRANSITION

The aggregate hides different agent behaviors, so walk through them one by one.

SLIDE 14

Claude Opus 4.7

POINT

Claude is the clean trust-and-follow case.

TALK TRACK

Claude consumes the pack as authority. With structured context, it opens fewer files, makes fewer tool calls per correct answer, and reaches perfect correctness in the current rerun. This is the clearest example of the completeness contract doing its job.

TRANSITION

Cursor's stronger model shows a different benefit: not just correctness, but speed.

SLIDE 15

Cursor Opus Medium

POINT

Structured context turns slow reconnaissance into targeted verification.

TALK TRACK

Bare Cursor Opus medium spends time on glob and grep before it knows where to read. With the pack, it still verifies, but it gets to the right area faster. The key story is the 219 seconds to 78 seconds duration drop, plus risk flags going to zero.

TRANSITION

Now contrast that with the default faster Cursor model.

SLIDE 16

Cursor Composer-2-Fast

POINT

The pack lifts the default fast model, but does not make it risk-free.

TALK TRACK

This is a useful practical slide because many people use the faster default model. The context pack gives the biggest correctness lift here, from 61 percent to 81 percent. But composer remains the riskiest lane, so the honest message is: structure helps weaker or faster models, but human review still matters.

TRANSITION

Codex shows the search-and-verify pattern even more explicitly.

SLIDE 17

Codex CLI

POINT

Codex reads the pack thoroughly and trades wall-clock time for safer answers.

TALK TRACK

Codex is not the speed story. It is the thoroughness story. It reads almost the whole pack, uses verification shortcuts more than other lanes, eliminates risk flags, and improves correctness modestly. The cost is slower structured runs. That is fine; the point is not that every metric always improves, but that the work becomes more controlled.

TRANSITION

Before anyone treats the chart as gospel, disclose the limits.

SLIDE 18

Methodology Disclosure

POINT

Trust comes from naming the limits.

TALK TRACK

Be direct: 288 graded answers, six repos, four model variants, bare versus structured. The May rerun is LLM-provisional, Cursor telemetry has gaps, and anomalies were preserved rather than hidden. The stance is not "perfect benchmark." The stance is "transparent measurement of a practical workflow."

TRANSITION

Now answer the predictable objection from a Cursor meetup room.

Prior Art

POINT

This is not a replacement for `.cursorrules`, MCP, vector search, or project memory.

TALK TRACK

Position `agent-context` as the navigation layer. `.cursorrules`, `AGENTS.md`, and `CLAUDE.md` become routing blocks into the pack. MCP is access, not navigation. Vector search is semantic recall; the pack is structural recall with contracts and verification shortcuts. These tools can coexist.

Also separate it from skills. Skills are best for highly repeatable tasks, organization knowledge, brand preferences, and taste: "how we write launch copy," "how we review a deck," "how this team scores social relevance." `agent-context` is the system map for a specific repo or system. The skill creates and maintains the map; the map is what every future agent session reads.

TRANSITION

Then be honest about what it costs.

Tradeoffs

POINT

The bargain is maintenance effort for less repeated rediscovery.

TALK TRACK

Name the costs plainly: the pack has to stay fresh, it can be overfit if it quotes answers instead of routing work, and not every repo needs the full tier. It does not replace review. It makes the first draft and the investigation path better.

Say the warning directly: bad context is worse than no context. If the map is stale or too confident, it can send the agent in the wrong direction faster. That is why freshness, scoped updates, and human review are not bureaucracy; they are the safety rails that make the context worth trusting.

TRANSITION

So what should people actually do tomorrow?

Try This

POINT

Start with one repo and one painful workflow.

TALK TRACK

Give the audience a small action with the actual commands on screen: `uv tool install agent-context-cli`, then `agent-context install-skill --agent claude`. After that it is one prompt to your coding agent — use the agent-context skill to build the pack — then review the generated diff, run verify and freshness, and test it on one workflow that usually causes confusion. As of this talk the package is live on PyPI; mention that briefly so the audience knows it is not vapor.

TRANSITION

Close by restating the principle, not the tool.

Closing

POINT

Make context part of the system.

TALK TRACK

End with the standard: context should not live only in a chat thread, a prompt, or one developer's head. For developers, it starts in the repo. For teams, it becomes a reviewable system artifact. The ask is simple: pick one repo, build the context pack, verify it, and open a PR.

FINAL LINE

Make the map part of the system, so every agent does not have to rediscover it from scratch.